Python Code

```python
#Fibonacci (DP – Bottom-Up)
def fibonacci(n):
    dp = [0]*(n+1)
    dp[0] = 0
    dp[1] = 1

    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]

print(fibonacci(10))
#Fibonacci (Top-Down Memoization)
def fib(n, memo={}):
    if n <= 1:
        return n

    if n in memo:
        return memo[n]

    memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]

print(fib(10))
#3. Climbing Stairs (similar to Fibonacci)
def climb_stairs(n):
    dp = [0]*(n+1)
    dp[0] = 1
    dp[1] = 1

    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]

print(climb_stairs(5))
#4. 0/1 Knapsack (Classic Beginner DP)
##Weights + values → maximize profit.
def knapsack(W, wt, val, n):
    dp = [[0]*(W+1) for _ in range(n+1)]

    for i in range(n+1):
        for w in range(W+1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif wt[i-1] <= w:
                dp[i][w] = max(
                    val[i-1] + dp[i-1][w-wt[i-1]],
                    dp[i-1][w]
                )
            else:
                dp[i][w] = dp[i-1][w]
```

```python

    return dp[n][W]

print(knapsack(7, [1,3,4,5], [1,4,5,7], 4))
#5. Subset Sum (DP Table)
def subset_sum(arr, target):
    n = len(arr)
    dp = [[False]*(target+1) for _ in range(n+1)]

    for i in range(n+1):
        dp[i][0] = True

    for i in range(1, n+1):
        for s in range(1, target+1):
            if arr[i-1] <= s:
                dp[i][s] = dp[i-1][s] or dp[i-1][s-arr[i-1]]
            else:
                dp[i][s] = dp[i-1][s]

    return dp[n][target]

print(subset_sum([2,3,7,8,10], 11))
#6. Coin Change (minimum coins)
def coin_change(coins, amount):
    dp = [float("inf")]*(amount+1)
    dp[0] = 0

    for coin in coins:
        for i in range(coin, amount+1):
            dp[i] = min(dp[i], dp[i-coin] + 1)

    return dp[amount] if dp[amount] != float("inf") else -1

print(coin_change([1,2,5], 11))
#7. Longest Increasing Subsequence (LIS) – O(n²)
def lis(arr):
    n = len(arr)
    dp = [1]*n

    for i in range(n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)

print(lis([10, 22, 9, 33, 21, 50]))
# 8. Longest Common Subsequence (LCS)
def lcs(x, y):
    n = len(x)
    m = len(y)
    dp = [[0]*(m+1) for _ in range(n+1)]

    for i in range(1, n+1):
        for j in range(1, m+1):
```

```
            if x[i-1] == y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[n][m]

print(lcs("abcde", "ace"))
# ⭐ 9. Edit Distance (Levenshtein)
def edit_distance(a, b):
    n, m = len(a), len(b)
    dp = [[0]*(m+1) for _ in range(n+1)]

    for i in range(n+1):
        dp[i][0] = i
    for j in range(m+1):
        dp[0][j] = j

    for i in range(1, n+1):
        for j in range(1, m+1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(
                    dp[i-1][j],
                    dp[i][j-1],
                    dp[i-1][j-1]
                )

    return dp[n][m]

print(edit_distance("horse", "ros"))
#⭐ 10. House Robber (cannot take adjacent)
def rob(nums):
    if len(nums) == 1:
        return nums[0]

    dp = [0]*len(nums)
    dp[0] = nums[0]
    dp[1] = max(nums[0], nums[1])

    for i in range(2, len(nums)):
        dp[i] = max(nums[i] + dp[i-2], dp[i-1])

    return dp[-1]

print(rob([1,2,3,1]))

#####################################33
dic = {}
def subsets(lst,sums):

    if sums == 0:
        return True
    if len(lst) == 0 or  sums<0:
```

```
            return False
        key = (tuple(lst),sums)
        if key in dic:
            return dic[key]

        include = subsets(lst[1:],sums-lst[0])
        exclude = subsets(lst[1:],sums)
        result =  (include or exclude)
        dic[key] = result

        return result

lst = [5,3,11,8,2]
sums = 16
result = subsets(lst,sums)
print(result)
lst = [2,3,34,11,1]
sums = 6
dp = []
ns = len(lst)
for i in range(ns+1):
    row = []
    for j in range(sums+1):
        if j == 0 :
            row.append(True)
        else:
            row.append(False)
    dp.append(row)
for i in dp:
    print(i)
print("_____________________________________________________________________")
n = len(dp)
for i in range(1,n):
    for j in range(1,sums+1):
        results = False
        if dp[i-1][j] == True:
            results = True
        else:
            if j-lst[i-1]>=0 and dp[i-1][j-lst[i-1]] == True:
                results = True


        dp[i][j]= results

for i in dp:
    print(i)
```